Network Security and Protocol Analysis Report

Case Study Report
Date: 23th April 2026

Section A: Firewall and Network Architecture

Section B: Protocol Analysis and HTTP Security

Section A: FIREWALL AND NETWORK ARCHITECTURE

1.1 Overview:

A mid-sized organization implemented a perimeter firewall using packet filtering and stateful inspection techniques to control network traffic through commonly used service ports such as HTTP and HTTPS. Despite these protections, attackers successfully bypassed the firewall by embedding malicious payloads within legitimate encrypted traffic that could not be inspected at the application layer. Since packet filtering firewalls analyze only header information and stateful firewalls allow trusted session-based communication, the malicious activity remained undetected. The absence of deep packet inspection, SSL traffic monitoring, and internal network segmentation further enabled attackers to gain unauthorized access to internal servers. As a result, sensitive customer data was exfiltrated through permitted outbound channels, highlighting limitations of traditional firewall architectures against modern cyber threats.

Key findings from the post-mortem investigation are as listed below:

- The firewall was a basic packet-filtering system using static access rules.
- No deep packet inspection (DPI) was active at the network edge.
- Internal server-to-server (east-west) traffic lacked monitoring.
- Attackers leveraged HTTPS sessions to deliver hidden malicious payloads.
- The breach remained undetected for approximately 47 days.

This suggests weaknesses in firewall configuration and architecture design.

1.2 Limitations of Basic Packet Filtering-Based Firewall Protection

Packet filtering operates at the **Network Layer (Layer 3)** and **Transport Layer (Layer 4)** of the OSI model. It evaluates individual data packets using predefined rule sets based on parameters such as source IP address, destination IP address, protocol type, and port number. However, this mechanism does not provide visibility into application-layer data or maintain awareness of session-level communication context. As a result, packet filtering firewalls are unable to detect malicious content embedded within otherwise legitimate network traffic.

Capability	Packet Filter Behaviour	Attack Vector Exploited
State Awareness	Each packet judged independently	Attackers injected forged TCP packets mid-session
Application Visibility	None	Malicious commands hidden inside valid HTTPS Traffic

Fragmentation Handling	Poor	Payload split across fragments to bypass rules
Spoofing Resistance	Low	IP Spoofing used to pass source-IP whitelisting
Logging Depth	Minimal	No audit trail of actual data accessed

1.3 Stateful Inspection: Strengths and Limitations

The stateful inspection is also referred to as *dynamic packet filtering.* It is the type of firewall technology that monitors the state of active connections and uses the information to permit the network packets through the firewall. Stateful inspection is generally used in place of stateless inspection of static packet filtering and is well suited with Transmission Control Protocol (TCP) and similar protocols, although it can also support protocols like User Datagram Protocol (UDP).

The key areas where stateful inspection firewalls fall short are:

Limitations of Stateful Inspection Firewalls

- **Lack of deep application-layer inspection**
  Stateful firewalls track session states but cannot analyze application-layer payloads such as HTTP request contents, allowing malicious commands hidden within legitimate traffic to pass undetected.
- **Limited visibility into encrypted traffic**
  These firewalls generally cannot inspect SSL or TLS encrypted communication without additional inspection modules, enabling attackers to exfiltrate sensitive data through secure channels.
- **Inability to detect zero-day attacks**
  Stateful inspection primarily relies on predefined rules and known connection patterns, which makes it ineffective against previously unknown attack techniques.
- **Weak detection of insider threats**
  Traffic originating from within trusted internal sessions is often assumed to be legitimate, allowing compromised internal systems to communicate with external malicious servers.
- **Insufficient monitoring of lateral movement**
  Stateful firewalls focus mainly on validating perimeter sessions and do not effectively monitor east–west traffic between internal network segments.

In the analyzed case study, the organization's stateful inspection firewall successfully monitored and maintained records of active HTTPS sessions, but it was unable to examine the encrypted contents of the transmitted data. The attacker exploited this limitation by establishing a legitimate TLS session from a trusted partner IP address, allowing malicious communication to

occur within an apparently authorized connection. As a result, the firewall permitted the traffic without raising security alerts, which enabled unauthorized activity to proceed undetected within the network environment.

1.4 Recommended Layered Firewall Architecture for Strengthened Network Protection

To effectively defend against advanced cyber threats, organizations should implement a layered security approach that combines multiple protective technologies across different levels of the network infrastructure. This approach includes the deployment of **Next Generation Firewalls (NGFWs)** for deep packet inspection and application-level visibility, **Web Application Firewalls (WAFs)** to protect web-based services from application-layer attacks, robust endpoint protection mechanisms to secure user devices, and centralized monitoring systems for continuous traffic analysis and threat detection.

Next-Generation Firewall (NGFW)

A Next-Generation Firewall operates at Layers 3-7 and incorporates deep packet inspection, application identification (App-ID), user identity awareness, and integrated intrusion prevention (IPS). Leading enterprise solutions include:

Product	Vendor	Key Differentiator
Palo Alto PA-Series	Palo Alto Networks	App-ID engine identifies 4000+ applications regardless of port/protocol
Cisco Firepower 4100	Cisco	TALOS threat intelligence integration + snort IPS Engine
Fortinet FortiGate	Fortinet	SPU hardware acceleration for high-throughout DPI at wire speed
Check Point Quantum	Check Point	Unified management with SandBlast zero-day threat emulation
Sophos XGS	Sophos	Synchronized Security links endpoint and network threat data

Web Application Firewall (WAF)

A WAF specifically protects HTTP/HTTPS applications by inspecting request and response payloads for OWASP Top 10 attack patterns, including SQL injection, cross-site scripting (XSS), and HTTP request smuggling. Recommended solutions:

Product	Deployment Mode	Best For
AWS WAF	Cloud-native	AWS-hosted applications with serverless rule sets
Cloudflare WAF	Reserve proxy/CFN	DDoS mitigation + bot management + WAF in one
F5 BIG-IP ASM	On-premise/hybrid	Complex enterprise apps needing granular policy control
ModSecurity (OWASP CRS)	Open-source/embedded	Cost-effective WAF for Apache/Nginx web servers
Imperva Cloud WAF	Cloud/SaaS	Advanced bot protection and API security

Recommended Layered Architecture

Layer	Technology	Function
Perimeter	NGFW (Palo Alto/Fortinet)	Block known threats, App-ID, IPS, TLS inspection
DMZ	WAF (Cloudflare/F5)	Protect web applications from OWASP attacks
Internal	Micro-segmentation (Zero Trust)	Limit lateral movement, east-west traffic control
Endpoint	EDR (CrowdStrike/SentinelOne)	Detect malware, ransomware at host level
Monitoring	SIEM (Splunk/Microsoft Sentinel)	Correlate events, enable rapid incident response

Section B: Protocol Analysis Case Study

2.1 Overview:

A financial web platform was compromised due to insecure HTTP configurations. The attacker exploited the absence of HSTS and manipulated HTTP headers to perform SSL stripping and session hijacking. Over 2,000 user sessions were compromised during a 3-week window.

Attack timeline and method:

- Attacker positioned on a shared network (hotel Wi-Fi at a finance conference).
- Intercepted HTTP redirect responses from the application (301/302 responses to HTTPS).
- SSL stripping tool (SSLstrip) replaced HTTPS links with HTTP, keeping plaintext connections alive.
- Injected malicious headers into server responses to manipulate client behaviour.
- Session tokens, transmitted in plaintext cookies, were captured and replayed.
- The absence of HSTS meant browsers had no cached policy to enforce HTTPS.

2.2 Deep Dive - HTTP Header Manipulation

HTTP headers are metadata fields transmitted with every request and response. Attackers who can position themselves as a man-in-the-middle (MITM) or exploit server-side injection vulnerabilities can modify these headers to alter application behaviour, steal credentials, or execute cross-site attacks.

Headers commonly exploited:

Header	Normal Purpose	How Attackers Abuse It
Host	Identify the target virtual host	Host header injection to route requests to attacker-controlled servers
X-forwarded-for	Pass client IP through proxies	Bypass IP-based access controls by spoofing trusted IPs
Referer	Indicate origin of request	Leak sensitive URL parameters; CSRF token exposure

Content-Type	Describe body format	MIME sniffing attacks - trick browser into executing injected scripts
Transfer-Encoding	Describe body encoding	HTTP Request Smuggling - desync front-end and back-end parsing
Set-Cookie	Issue session cookies	Inject malicious cookies to override legitimate session state
Location	Redirect to a new URL	Open redirect attacks - redirect users to phishing sites

HTTP Request Smuggling - Technical Analysis

HTTP Request Smuggling (HRS) exploits discrepancies between how front-end proxies and back-end servers parse the Transfer-Encoding and Content-Length headers. When both headers are present, different systems may disagree on where one request ends and another begins, allowing an attacker to 'smuggle' a partial request that is prepended to the next legitimate user's request.

Example of a CL.TE attack (Content-Length used by front-end, Transfer-Encoding by back-end):

POST / HTTP/1.1 Host: vulnerable-app.com Content-Length: 13 Transfer-Encoding: chunked 0 GET /admin HTTP/1.1

2.3 HSTS — What It Is and Why Its Absence Was Critical

HTTP Strict Transport Security (HSTS) is a web security policy mechanism defined in RFC 6797. When an HSTS header is served by a web application, the browser caches a policy that forces all future connections to that domain to use HTTPS — even if the user types 'http://' or clicks an HTTP link. This cached policy prevents SSL stripping attacks because the browser enforces HTTPS before any network request is made.

Correct HSTS header implementation:

Strict-Transport-Security: max-age=31536000; includeSubDomains; preload

HSTS Directive	Meaning	Recommendation
max-age=31536000	Browser caches HSTS policy for 1 year (in seconds)	Minimum 6 months; 1 year is standard
includeSubDomains	Policy applies to add subdomains	Required - without this, subdomains can be stripped
proload	Submit domain to browser preload list	Recommended for maximum protection before first visit

In the case study, the application served no HSTS header whatsoever. The attacker's SSLstrip tool successfully downgraded all connections because the victims' browsers had no cached HTTPS enforcement policy.

2.4 Prevention Measures — Comprehensive Recommendations

A. Enforce HTTPS and HSTS

Deploy and configure HSTS with max-age of at least 31,536,000 (1 year), includeSubDomains, and preload.

Submit the domain to the HSTS Preload List at hstspreload.org — browsers ship with this list built in.

Configure TLS 1.2 minimum (TLS 1.3 preferred); disable SSLv3, TLS 1.0, and 1.1.

Use a strong cipher suite: prefer ECDHE for forward secrecy; disable RC4, DES, and 3DES.

B. Secure HTTP Response

Security Header	Recommended Value	Protection Provided
Content-Security-Policy	default-src ‘self’; script-src ‘self’	Prevents XSS and data injection attacks
X-Content-Type-Options	nosniff	Prevents MIME-type sniffing attacks
X-Frame-Options	DENY or SAMEORIGIN	Prevents clickjacking via iframe embedding

X-XSS-Protection	1; mode=block	Legacy XSS Filter (support modern CSP instead)
Referrer-Policy	strict-origin-whenn-cross-origi n	Controls referrer data leakage
Permissions-Policy	geolocation=(), microphone=()	Restricts brows feature access

C. Prevent HTTP Request Smuggling

Ensure front-end proxy and back-end server use the same HTTP specification version (prefer HTTP/2 end-to-end).

Configure web servers to reject requests containing both Content-Length and Transfer-Encoding headers.

Use a WAF with HTTP normalisation capabilities (e.g., F5 BIG-IP ASM, Cloudflare WAF).

Enable HTTP/2 — its binary framing protocol makes request smuggling significantly harder.

Regularly run Burp Suite's HTTP Request Smuggler scanner against all endpoints.

D. Cookie Security

Set the Secure flag on all session cookies — prevents transmission over unencrypted HTTP.

Set the HttpOnly flag — prevents JavaScript from accessing session cookies.

Use SameSite=Strict or SameSite=Lax to prevent cross-site request forgery via cookie leakage.

Implement short session expiry and token rotation after authentication events.

E. Network-Level Controls

Deploy a TLS-terminating reverse proxy (e.g., Nginx, HAProxy, Cloudflare) that normalises all requests.

Implement mutual TLS (mTLS) for service-to-service communication inside the infrastructure.

Use certificate pinning for mobile applications to prevent rogue certificate acceptance.

Enable OCSP stapling for certificate revocation checking without leaking user browsing data.

Vulnerability	Attack Technique	Prevention Measure	Priority
No HSTS	SSL Stripping (SSLstrip)	Deploy HSTS+Preload list	Critical
Missing Secure cookie flag	Cookie theft over HTTP	Secure+HTTPOnly+SameSite	Critical
HTTP header injection	Response splitting, redirect abuse	WAP+Input sanitisation	High
Request smuggling	CL.TE/TE.CL desync	HTTP/2 end-to-end + WAF normalisation	High
Weak TLS Configuration	BEAST, POODLE, CRIME Attacks	TLS 1.3, strong cipher suites only	High
No CSP Header	Cross-site scripting (XSS)	Strict Content-Security-Policy	Medium
Missing X-Frame-Options	Clickjacking	X-Frame-Options: DENY	Medium

Conclusion:

Both case studies demonstrate that no single security control is sufficient in isolation. The data breach in Section 1 occurred because packet-filtering firewalls were trusted as the sole gatekeepers, while attackers exploited the absence of application-layer visibility. The HTTP attack in Section 2 succeeded because a missing HSTS header and improper response-header controls created exploitable gaps at the protocol level.

Effective network security requires a layered defence strategy — combining NGFWs, WAFs, proper HTTP security headers, strong TLS configuration, and continuous monitoring — to address threats at every layer of the OSI stack. Security is not a product you buy; it is a practice you continuously maintain.